

Programación 3

Trabajo Práctico Especial

Cursada 2026

Contexto

El trabajo práctico de cursada propone resolver distintos problemas asociados a un contexto simplificado de **paquetes y camiones de reparto**. En una primera etapa se deberán resolver servicios simples de búsqueda y consulta sobre los paquetes, mientras que en la segunda etapa el objetivo estará centrado en resolver la asignación de paquetes a camiones, utilizando distintas técnicas algorítmicas.

Para ambas etapas el conjunto de datos de entrada estará dado por **2 archivos de texto** con un formato predefinido, que se menciona a continuación.

```
Camiones.csv
<camiones_totales>
<id_camion>;<patente>;<esta_refrigerado>;<capacidad_kg>
```

```
Paquetes.csv
<paquetes_totales>
<id_paquete>;<codigo_paquete>;<peso_kg>;<contiene_alimentos>;<nivel_urgencia>
```

Un **camión** está determinado por un ID único, una patente, si está refrigerado o no, y su capacidad máxima de carga expresada en kilogramos.

Un **paquete** está determinado por un ID único, un código identificador, su peso en kilogramos, si contiene alimentos o no, y un nivel de urgencia (valor entero entre 1 y 100).

Por ejemplo:

```
Camiones.csv
3
100;AAA000A;1;100
101;AAA001B;0;500
102;AAA002C;1;115
```

```
Paquetes.csv
4
1;P001;30;1;80
2;P002;100;0;2
3;P003;80;0;10
4;P004;25;1;100
```

El primer problema a resolver es implementar un algoritmo que reciba el nombre de ambos archivos .csv, los procese y cargue la información obtenida, para que luego pueda ser utilizada por los distintos algoritmos a implementar.

Primera Parte

El objetivo de la primera parte es plantear una resolución eficiente para los siguientes servicios:

- **Servicio 1:** Dado un código de paquete (String), retornar toda la información del paquete asociado. En caso de no existir, retornar null.
- **Servicio 2:** Dado un booleano que indica si se buscan paquetes que contienen alimentos (true) o que no contienen alimentos (false), retornar el listado de paquetes correspondiente.
- **Servicio 3:** Dados dos valores enteros que representan un nivel de urgencia mínimo y máximo, retornar todos los paquetes cuyo nivel de urgencia se encuentre dentro de ese rango (inclusive).

Se establece la siguiente clase para la resolución de los servicios, la cuál deberá ser utilizada **sin modificar la interfaz de los métodos públicos** pero pudiendo agregar los métodos y estructuras privadas que consideren apropiadas.

```
public class Servicios {  
    //Completar con las estructuras y métodos privados que se  
    requieran.  
  
    /*  
     * Expresar la complejidad temporal del constructor.  
     */  
    public Servicios(String pathCamiones, String pathPaquetes)  
{  
  
    }  
  
    /*  
     * Expresar la complejidad temporal del servicio 1.  
     */  
    public Paquete servicio1(String codigoPaquete) { }  
  
    /*  
     * Expresar la complejidad temporal del servicio 2.  
     */  
    public List<Paquete> servicio2(boolean contieneAlimentos) {  
    }  
  
    /*  
     * Expresar la complejidad temporal del servicio 3.  
     */  
    public List<Paquete> servicio3(int urgenciaMinima, int  
    urgenciaMaxima) { }  
}
```

Segunda Parte

Se desea establecer una **asignación de todos los paquetes a los camiones disponibles** con el objetivo de **minimizar el peso total de los paquetes que no pudieron ser asignados a ningún camión**.

Se sabe que existen ciertas restricciones para asignar un paquete a un camión:

- **Primero**, ningún camión podrá superar su **capacidad máxima de carga**. La capacidad de cada camión está definida en el archivo de entrada.
- **Segundo**, los paquetes que **contienen alimentos** sólo podrán ser asignados a camiones **refrigerados**.

Objetivo

El objetivo de esta segunda parte es resolver el problema planteado mediante dos técnicas algorítmicas distintas: **Backtracking** y **Greedy**. Luego se deberán presentar los resultados teniendo en cuenta distintas métricas que permitan visualizar, mínimamente, la calidad de la solución y el costo de obtener dicha solución con ambas técnicas.

Para la presentación de resultados se propone el siguiente formato:

```
Backtracking
Solución obtenida: cada camión con los paquetes asignados.
Peso no asignado: <peso total de paquetes sin asignar> kg.
Métrica para analizar el costo de la solución (cantidad de
estados generados).

Greedy
Solución obtenida: cada camión con los paquetes asignados.
Peso no asignado: <peso total de paquetes sin asignar> kg.
Métrica para analizar el costo de la solución (cantidad de
candidatos considerados).
```

También se solicita una **breve explicación de la estrategia elegida** para resolver el problema mediante cada técnica. Para esto, se propone acompañar la implementación con un breve comentario antes de la función principal, por ejemplo:

```
/*
 * <<Breve explicación de la estrategia de resolución>>
 */
public Solucion backtracking(...) { }
```

Requisitos de la entrega

Se deberá entregar un **proyecto que compile correctamente** con el código fuente de la aplicación solicitada. La entrega se realizará por email a la cuenta de la materia, adjuntando el código fuente en un archivo comprimido o compartiendo un link a un repositorio de Git.

La fecha de entrega límite se establece para el **viernes 19/6 hasta las 23:59 hs.** En las semanas posteriores se realizará una devolución del trabajo entregado permitiendo, de ser necesario, una re-entrega con las correcciones requeridas.

Importante: no se aceptarán entregas fuera de término.

Criterios de evaluación

Se tendrán en cuenta los siguientes aspectos en la corrección:

- **Correctitud:** los servicios deben retornar los resultados esperados.
- **Eficiencia:** las estructuras de datos utilizadas deben justificar la complejidad temporal declarada en los comentarios.
- **Implementación de Backtracking:** el algoritmo debe explorar el espacio de soluciones correctamente, respetando las restricciones.
- **Implementación de Greedy:** el algoritmo debe definir claramente su función de selección y justificarlo en el comentario.
- **Calidad del código:** legibilidad, uso apropiado de clases y métodos, y ausencia de código redundante.

Consideraciones adicionales

- Se pueden utilizar las estructuras de datos de la librería de Java (HashMap, ArrayList, etc.).
- Cualquier duda sobre el enunciado deberá consultarse antes de la fecha de entrega.